Design and Implementation of an Efficient Priority-Based Emergency Patient Management System using Appropriate Data Structures for Severity-Based Treatment and Arrival-Order Processing

`ASSIGNMENT REPORT

Submitted in the partial fulfilment for the Course

of

CSA0307- Data Structures

to the award of the degree of

BACHELOR OF ENGINEERING

IN

BTECH IN COMPUTERSCIENCE

Submitted by

Devadharshini M(192572149)

Shamah Ali Fathima(192573002)

Blessika S(192525251)

Under the Supervision of

Dr.K.Kiruthika

Saveetha School of engineering

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

September 2026

Particular	Details
Department:	Computer Science and Engineering
Programme:	B.Tech
Course Code & Course Name	CSA0307 Data Structure
Academic Year / Batch	2nd year (2025-2029)
Faculty Name	Dr.K.Kiruthika
Assignment Title	Design and implementation of an efficient priority-based emergency patient management system using appropriate data structures for severity-based treatment and arrival-order processing
Date of Issue	31-08-2026
Date of Submission	02-09-2026
Maximum Marks	100
Course Outcome(s) – CO	CO2: Apply the suitable Abstract Data type for construction of the smart city emergency system CO4: Make use of heaps, priority queue and hashing techniques for graph storage and route cache for repeated queries in C. CO5: Develop robust smart city emergency system by implementing shortest path algorithms such as Dijkstra's algorithm, Floyd-Warshall.
Bloom's Taxonomy Level	L4 – Analyze

SDG Mapping (SDG 1-17)	SDG-9 and SDG-11

DESIGN AND IMPLEMENTATION OF AN EFFICIENT PRIORITY-BASED EMERGENCY PATIENT MANAGEMENT SYSTEM USING APPROPRIATE DATA STRUCTURES FOR SEVERITY-BASED TREATMENT AND ARRIVAL-ORDER PROCESSING

B. Assignment Problem / Challenge

Design and implement an efficient **Priority-Based Emergency Patient Management System** that manages patients according to their medical severity while maintaining the arrival order among patients having the same severity.

The system should efficiently support patient registration, severity assignment, identification of the next patient for treatment, removal of treated patients, patient searching, and patient information updating.

The major challenge is to select an appropriate data structure that provides efficient priority-based treatment while preserving FIFO order among patients with equal severity.

C. Problem Statement

A hospital emergency department receives patients continuously throughout the day. Each patient may require a different level of medical attention depending on the severity of their condition.

For every patient, the system maintains:

- Unique patient ID
- Patient name
- Arrival time
- Severity level
- Assigned doctor
- Arrival sequence

Patients with critical or life-threatening conditions must be treated before patients with less severe conditions.

However, when two or more patients have the same severity level, they should generally be treated in the order in which they arrived, following the **First-In-First-Out (FIFO)** principle within that priority level.

For example, if three patients have severity level 5 and arrive in the order P101, P102, and P103, their treatment order should be:

P101 → P102 → P103

even though all three have the same severity.

The system must efficiently support patient insertion, priority assignment, selection of the next patient, deletion after treatment, searching, and updating.

Different data structures such as arrays, linked lists, simple queues, and priority queues can be considered. The most appropriate data structure must be selected based on efficiency, scalability, priority handling, and arrival-order requirements.

D. Requirements and Constraints

Functional Requirements

The system should:

- Register new patients.
- Store patient details.
- Assign severity levels.
- Maintain arrival order.
- Select the highest-priority patient.
- Remove patients after treatment.
- Search for a particular patient.
- Update patient information.
- Update assigned doctor information.
- Display waiting patients.

Performance Requirements

- Patient insertion should be efficient.
- The next patient should be identified quickly.

- Patient removal should be efficient.
- The system should handle increasing numbers of patients.

Technical Requirements

- Implement a Priority Queue.
- Use a suitable heap-based structure.
- Maintain severity as the primary priority.
- Maintain arrival order as the secondary priority.
- Analyze time and space complexity.

Safety Requirements

Critical patients should receive higher priority than less severe patients.

Reliability Requirements

The system should correctly maintain treatment order even when patients with different severity levels arrive at different times.

User Requirements

The interface should allow hospital staff to easily:

- Add patients
- View the next patient
- Treat patients
- Search patients
- Update information
- Display the queue

Security Requirements

In a real-world system, patient information should be protected using authentication, authorization, and secure data storage.

Scalability Requirements

The system should remain efficient when the number of waiting patients increases significantly.

E. Student Work

1. Problem Understanding and Formulation

1.1 What is the Problem?

The emergency department needs to manage patients according to medical urgency rather than simply according to their arrival time.

A conventional FIFO queue processes patients in arrival order:

P101 → P102 → P103 → P104

Suppose:

P101 → Severity 2
P102 → Severity 1
P103 → Severity 5
P104 → Severity 3

A FIFO queue would process:

P101 → P102 → P103 → P104

This is not appropriate because P103 has a critical condition.

The required treatment order is:

P103 → P104 → P101 → P102

Therefore, the system requires a **Priority Queue**.

1.2 Expected Outcomes

The proposed system should:

1. Register emergency patients.
2. Store patient details.
3. Assign severity-based priority.

4. Select critical patients first.
5. Maintain FIFO order among equal-severity patients.
6. Remove patients after treatment.
7. Search patient information.
8. Update patient and doctor information.
9. Handle empty queues.
10. Provide efficient performance.
11. Scale to large patient loads.
12. Demonstrate the advantages of Priority Queue over FIFO Queue.

1.3 Available Information / Data

Data	Description
Patient ID	Unique identifier
Patient Name	Name of patient
Arrival Time	Time patient entered emergency department
Severity Level	Medical urgency from 1 to 5
Doctor	Assigned medical professional
Arrival Sequence	Order in which patients arrived
Status	Waiting / Treated

Severity Classification

Level	Description
5	Critical / Life-threatening
4	Very Serious
3	Moderate Emergency
2	Mild Emergency
1	Non-Critical

1.4 Assumptions

1. Every patient has a unique patient ID.
2. Severity ranges from 1 to 5.
3. Higher severity indicates higher priority.
4. Patients with equal severity follow FIFO order.
5. Arrival sequence numbers are unique.

6. Patients are removed after treatment.
7. Doctor information can be updated.
8. Patient information is maintained while the patient is waiting.
9. The system is a demonstration model and does not replace professional medical decisions.

1.5 Constraints

- Patient IDs must be unique.
- Severity must be within the allowed range.
- Critical patients should receive priority.
- Equal-severity patients must maintain arrival order.
- The system should correctly handle an empty queue.
- The system should support increasing patient numbers.
- Searching and updating should not corrupt the priority structure.

2. Application of Course Knowledge

2.1 Abstract Data Type – Priority Queue

A **Priority Queue** is an Abstract Data Type in which each element is associated with a priority.

Unlike a normal queue, the element with the highest priority is removed first.

For the emergency patient system:

Priority = Severity

and when severity is equal:

Priority = Arrival Order

2.2 Patient Data Structure

Each patient can be represented using a structure:

Patient
{
  Patient ID
  Patient Name
  Arrival Time
  Severity
  Assigned Doctor
  Arrival Sequence
}

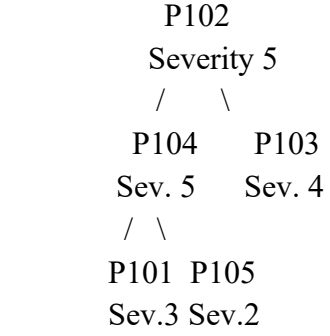
The arrival sequence acts as the tie-breaker.

2.3 Priority Queue

The Priority Queue stores all patients waiting for treatment.

The patient with the highest priority is maintained at the top.

Example:



The root contains the patient who should be treated next.

2.4 Binary Heap

A **Binary Max-Heap** can be used to implement the Priority Queue.

The heap maintains the following rule:

Every parent has a priority greater than or equal to its children.

For this application:

Higher Severity = Higher Priority

If severity is equal:

Earlier Arrival = Higher Priority

The Binary Heap provides efficient insertion and deletion.

2.5 Severity-Based Priority

The priority levels are:

Severity 5 → Highest Priority
Severity 4 → Very High Priority
Severity 3 → Medium Priority
Severity 2 → Low Priority
Severity 1 → Lowest Priority

Example:

P101 → Severity 2
P102 → Severity 5
P103 → Severity 4
P104 → Severity 3

Treatment order:

P102 → P103 → P104 → P101

2.6 Arrival-Order Processing

Arrival order is important when patients have equal severity.

Example:

P101 → Severity 5 → Arrival 1
P102 → Severity 5 → Arrival 2
P103 → Severity 5 → Arrival 3

Treatment order:

P101 → P102 → P103

This ensures fairness within the same severity level.

2.7 Queue Operations

The major Priority Queue operations are:

Insert

Adds a new patient.

Complexity: O(log n)

Peek

Returns the next patient without removing them.

Complexity: O(1)

Delete / Extract

Removes the highest-priority patient.

Complexity: O(log n)

IsEmpty

Checks whether patients are waiting.

Complexity: O(1)

2.8 Searching and Updating Patient Information

Searching for a specific patient by ID requires checking the stored patient records.

Worst-case search complexity:

O(n)

If the severity of a patient changes, the patient's position in the Priority Queue may also need to be adjusted.

2.9 Complexity

For **n waiting patients**:

Operation	Complexity
Insert	O(log n)
Peek	O(1)
Delete	O(log n)
Search	O(n)
Update	O(n) worst case
Display	O(n log n) if extracted in order
Space	O(n)

3. Solution / Design / Methodology

3.1 System Architecture

The proposed system consists of the following modules:

Patient Registration
↓
Patient Information
↓

Severity Assignment
↓
Arrival Sequence Assignment
↓
Priority Queue
↓
Highest Priority Patient
↓
Treatment
↓
Patient Removal

Additional modules:

Search Patient
Update Patient
Display Patients

3.2 System Workflow

START
↓
Register Patient
↓
Enter Patient Details
↓
Assign Severity
↓
Assign Arrival Sequence
↓
Insert into Priority Queue
↓
Select Highest Priority Patient
↓
Treat Patient
↓
Remove Patient
↓
Display Updated Queue

↓
END

3.3 Patient Registration Module

The registration module collects:

- Patient ID
- Patient name
- Arrival time
- Severity
- Assigned doctor

After registration, the system generates an arrival sequence number.

3.4 Severity Assignment Module

Severity is assigned between 1 and 5.

5 → Critical
4 → Very Serious
3 → Moderate
2 → Mild
1 → Non-Critical

Higher severity receives higher treatment priority.

3.5 Priority Queue Management

The Priority Queue maintains patients according to:

First Priority → Severity
Second Priority → Arrival Sequence

Thus:

Severity 5, Arrival 1

has higher priority than:

Severity 5, Arrival 2

3.6 Next Patient Selection

The root of the Max-Heap contains the patient with the highest priority.

Therefore, the system can identify the next patient efficiently in:

O(1)

3.7 Patient Treatment and Removal

After treatment:

1. The root patient is selected.
2. The patient is removed.
3. The last heap element is moved to the root.
4. Heapify-down is performed.
5. The heap property is restored.

Complexity:

O(log n)

3.8 Patient Search

The system searches for a patient using Patient ID.

Example:

Enter Patient ID: 103

Patient Found:

ID: 103
Name: Kavin
Severity: 4
Doctor: Dr. Ravi

3.9 Patient Information Update

The system can update:

- Severity
- Doctor
- Patient status

If severity changes, the Priority Queue must be reorganized to maintain the correct treatment order.

3.10 Algorithm / Pseudocode

Insert Patient

START

Read patient details

Check whether Patient ID already exists

```
IF ID exists
   Display "Patient already exists"
   STOP
END IF
```

Assign arrival sequence

Insert patient into Priority Queue

Restore heap property

Display "Patient added successfully"

END

Select Next Patient

START

IF Priority Queue is empty
  Display "No patients waiting"
  STOP
END IF

Access root of Priority Queue

Display patient information

END

Treat Patient

START

IF queue is empty
  Display "No patients waiting"
  STOP
END IF

Select highest-priority patient

Remove patient from Priority Queue

Restore heap property

Display treated patient

END

3.11 Conventional FIFO Queue vs Priority Queue

A conventional FIFO queue considers only arrival order.

Example:

Patient A → Severity 1
Patient B → Severity 2
Patient C → Severity 5

FIFO treatment:

A → B → C

Priority Queue treatment:

C → B → A

Therefore, Priority Queue is more appropriate for emergency patient management.

3.12 Data Structure Selection and Justification

The **Priority Queue using Binary Max-Heap** is selected because:

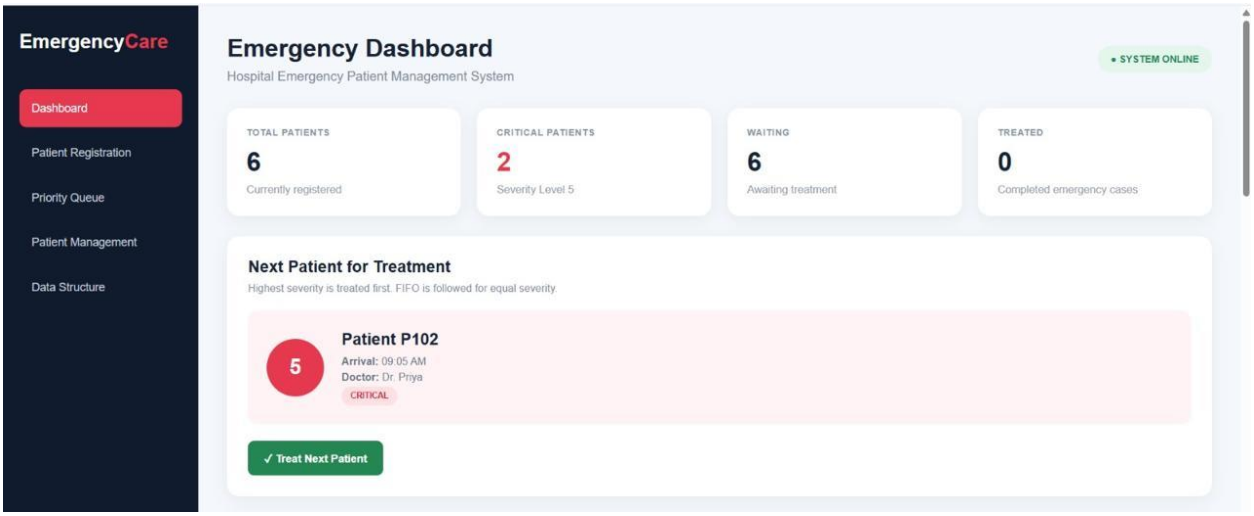
- It directly supports priority-based processing.
- The highest-severity patient is available at the root.
- Insertion is O(log n).
- Removal is O(log n).
- Next-patient selection is O(1).
- Arrival order can be incorporated as a secondary priority.
- It scales better than repeatedly scanning an unsorted list.

4. Use of Modern Tools

Tool / Technology	Purpose
C++	Main programming language
STL Priority Queue	Priority Queue implementation
Vector	Internal heap storage

Custom Comparator	Severity and arrival-order comparison
VS Code	Program development
Git	Version control
GitHub	Project repository and submission

5. Results and Validation



SOURCE CODE :

```
#include <iostream>

#include <queue>

#include <vector>

#include <string>

using namespace std;

struct Patient {

    int id;

    string name;

    string arrivalTime;

    int severity;
```

```
    string doctor;
    int arrivalOrder;
};

// Higher severity gets higher priority.
// If severity is same, earlier arrival gets higher priority.
struct Compare {
    bool operator()(const Patient& a, const Patient& b) {
        if (a.severity != b.severity)
            return a.severity < b.severity;
        return a.arrivalOrder > b.arrivalOrder;
    }
};
priority_queue<Patient, vector<Patient>, Compare> pq;
int arrivalCounter = 0;
// Add patient
void addPatient() {
    Patient p;
    cout << "\nEnter Patient ID: ";
    cin >> p.id;
    cout << "Enter Patient Name: ";
    cin >> p.name;
    cout << "Enter Arrival Time: ";
```

```
    cin >> p.arrivalTime;
    cout << "Enter Severity (1-5): ";
    cin >> p.severity;
    if (p.severity < 1 || p.severity > 5) {
        cout << "Invalid severity level!\n";
        return;
    }
    cout << "Enter Assigned Doctor: ";
    cin >> p.doctor;
    p.arrivalOrder = ++arrivalCounter;
    pq.push(p);
    cout << "Patient added successfully!\n";
}
// View next patient
void viewNextPatient() {
    if (pq.empty()) {
        cout << "\nNo patients waiting.\n";
        return;
    }
    Patient p = pq.top();
    cout << "\nNext Patient for Treatment\n";
    cout << "Patient ID: " << p.id << endl;
    cout << "Name: " << p.name << endl;
```

```
    cout << "Arrival Time: " << p.arrivalTime << endl;
    cout << "Severity: " << p.severity << endl;
    cout << "Doctor: " << p.doctor << endl;}
// Treat patient
void treatPatient() {
    if (pq.empty()) {
        cout << "\nNo patients waiting.\n";
        return;
    }
    Patient p = pq.top();
    pq.pop();
    cout << "\nPatient Treated Successfully\n";
    cout << "Patient ID: " << p.id << endl;
    cout << "Name: " << p.name << endl;
    cout << "Severity: " << p.severity << endl;
}
// Search patient
void searchPatient() {
    if (pq.empty()) {
        cout << "\nNo patients waiting.\n";
        return;
    }
    int id;
```

```
    cout << "\nEnter Patient ID to search: ";
    cin >> id;
    priority_queue<Patient, vector<Patient>, Compare> temp = pq;
    bool found = false;
    while (!temp.empty()) {
        Patient p = temp.top();
        temp.pop();
        if (p.id == id) {
            cout << "\nPatient Found\n";
            cout << "Patient ID: " << p.id << endl;
            cout << "Name: " << p.name << endl;
            cout << "Arrival Time: " << p.arrivalTime << endl;
            cout << "Severity: " << p.severity << endl;
            cout << "Doctor: " << p.doctor << endl;
            found = true;
            break;
        }
    }
    if (!found)
        cout << "Patient not found.\n";
}
// Display all waiting patients
void displayPatients() {
```

```
    if (pq.empty()) {
        cout << "\nNo patients waiting.\n";
       return;   }
    priority_queue<Patient, vector<Patient>, Compare> temp = pq;
    cout << "\nPatients Waiting for Treatment\n";
    while (!temp.empty()) {
        Patient p = temp.top();
        temp.pop();
        cout << "\nPatient ID: " << p.id;
        cout << "\nName: " << p.name;
        cout << "\nSeverity: " << p.severity;
        cout << "\nArrival Order: " << p.arrivalOrder;
        cout << "\nDoctor: " << p.doctor << endl;
    }
}
// Update doctor information
void updateDoctor() {
    if (pq.empty()) {
        cout << "\nNo patients waiting.\n";
        return;
    }
    int id;
    string newDoctor;
```

```
    cout << "\nEnter Patient ID: ";
    cin >> id;
    cout << "Enter New Doctor: ";
    cin >> newDoctor;
    priority_queue<Patient, vector<Patient>, Compare> temp;
    bool found = false;
    while (!pq.empty()) {
        Patient p = pq.top();
        pq.pop();
        if (p.id == id) {
            p.doctor = newDoctor;
            found = true;
        }
        temp.push(p);
    }
    pq = temp;
    if (found)
        cout << "Doctor information updated successfully.\n";
    else
        cout << "Patient not found.\n";
}
int main() {
    int choice;
```

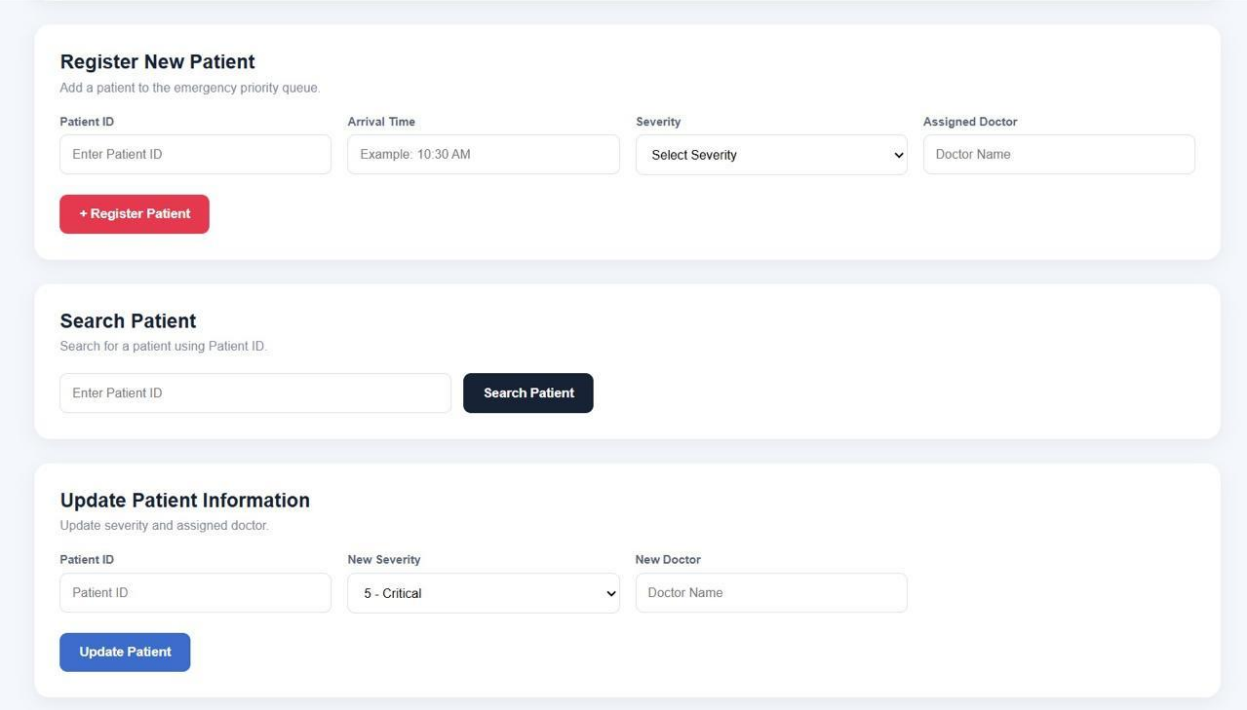
```
do {
    cout << "\n========================================";
    cout << "\n EMERGENCY PATIENT MANAGEMENT SYSTEM";
    cout << "\n========================================";
    cout << "\n1. Register Patient";
    cout << "\n2. View Next Patient";
    cout << "\n3. Treat Patient";
    cout << "\n4. Search Patient";
    cout << "\n5. Update Doctor";
    cout << "\n6. Display All Patients";
    cout << "\n7. Exit";
    cout << "\nEnter your choice: ";
    cin >> choice;
    switch (choice) {
        case 1:
            addPatient();
            break;
        case 2:
            viewNextPatient();
            break;
        case 3:
            treatPatient();
            break;
```

```
            case 4:
                searchPatient();
                break;
            case 5:
                updateDoctor();
                break;
            case 6:
                displayPatients();
                break;
            case 7:
                cout << "\nThank you!\n";
                Break;
            default:
                cout << "\nInvalid choice!\n";
        }
    } while (choice != 7);
    return 0;
}
```

Test Case 1: Different Severity Levels

Input:

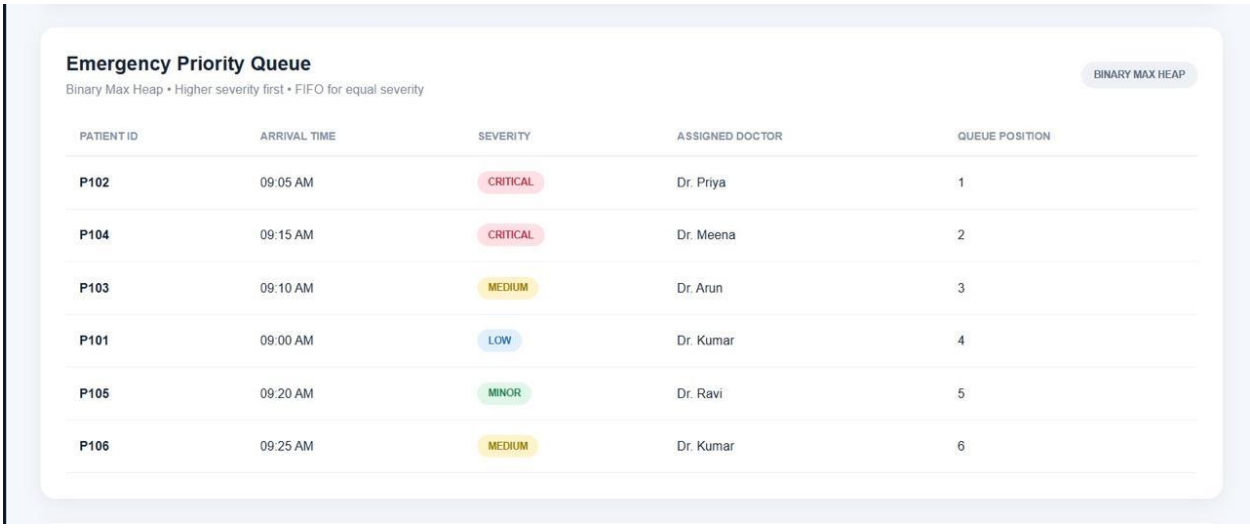
Four patients arrive at the emergency department. Patient P101 has severity level 2, P102 has severity level 5, P103 has severity level 3, and P104 has severity level 4. They arrive in the order P101, P102, P103, and P104.



Test Case 2: Same Severity – FIFO Processing

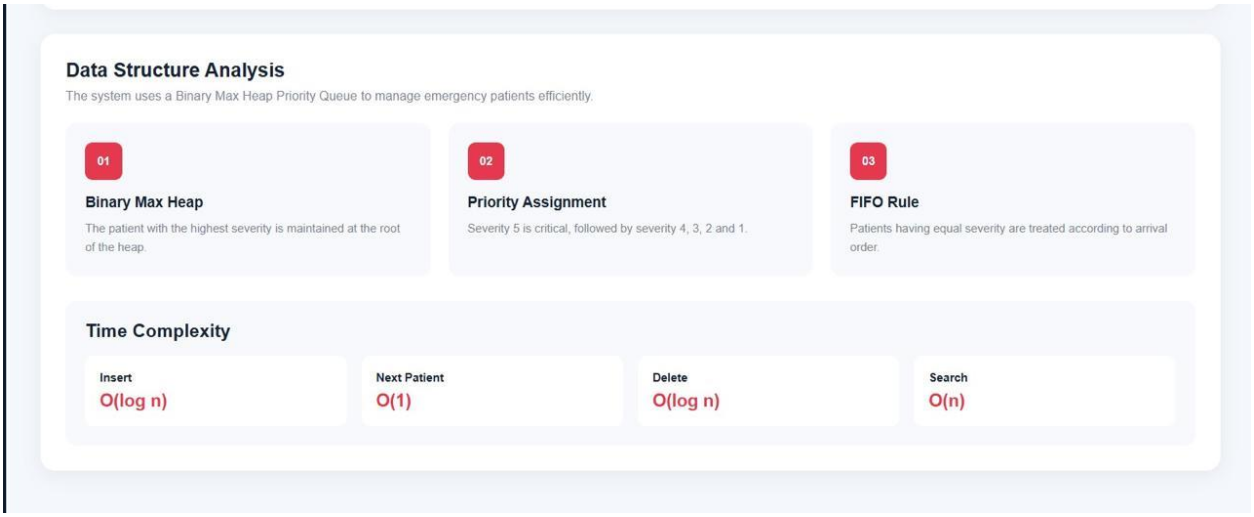
Input:

Three patients P201, P202, and P203 arrive with the same severity level of 5. They arrive in the order P201, P202, and P203. Another patient P204 arrives with severity level 3.



Test Case 3: Critical Patient Arrives Later

Input:
Initially, three patients are waiting: P301 with severity level 2, P302 with severity level 3, and P303 with severity level 4. Later, patient P304 arrives with severity level 5.



6. Analysis and Engineering Decision

6.1 Final Engineering Decision

The final solution uses:

Priority Queue + Binary Max-Heap + Arrival Sequence

This structure satisfies both major requirements:

Severity-based priority
+
FIFO among equal severity

6.2 Why Priority Queue Was Selected

A Priority Queue was selected because emergency patients cannot always be treated strictly according to arrival time.

Critical patients need higher priority.

The Binary Heap provides efficient operations while maintaining the required ordering.

6.3 Comparison with FIFO Queue

FIFO is simple but does not consider severity.

Therefore, a critical patient can be delayed by patients who arrived earlier but have less severe conditions.

Priority Queue avoids this problem.

6.4 Comparison with Array

An unsorted array requires scanning the array to identify the highest-priority patient.

This requires:

O(n)

The Priority Queue provides:

O(1) next-patient access.

6.5 Comparison with Linked List

A linked list provides dynamic storage but finding the highest-priority patient may require O(n).

The Binary Heap provides more efficient priority processing.

6.6 Time Complexity Analysis

Operation	Best	Average	Worst
Insert	O(1)	O(log n)	O(log n)
Peek	O(1)	O(1)	O(1)
Delete	O(log n)	O(log n)	O(log n)
Search	O(1)	O(n)	O(n)
Update	O(1)*	O(n)	O(n)

*If the required record is immediately available; locating an arbitrary patient may require O(n).

6.7 Space Complexity Analysis

The Priority Queue stores n patients.

Therefore:

Space Complexity = O(n)

6.8 Best-Case, Average-Case and Worst-Case Analysis

Best Case

If a newly inserted patient already satisfies the heap property, insertion may require only constant work.

Average Case

Insertion and deletion generally require logarithmic heap adjustment.

Worst Case

An inserted patient may move from a leaf to the root, requiring:

O(log n)

Similarly, deleting the root may require moving an element down the heap:

O(log n)

6.9 Scalability Analysis

The Binary Heap is suitable for large patient queues because insertion and removal are logarithmic.

For example, even when the number of waiting patients increases from hundreds to thousands, the number of heap operations grows logarithmically rather than requiring a complete scan for every insertion or removal.

Algorithm Comparison

Data Structure	Advantage	Limitation	Suitability
FIFO Queue	Simple and fast	Ignores severity	Low
Array	Easy implementation	Priority search O(n)	Medium
Linked List	Dynamic storage	Priority search O(n)	Medium
Sorted Array	Fast highest-priority access	Insertion can be O(n)	Medium
Priority Queue + Binary Heap	Efficient priority processing	Slightly more complex	**High**

7. Broader Considerations

7.1 Safety

The system prioritizes critically ill patients, reducing the possibility that a critical case will wait behind less severe cases simply because it arrived later.

However, real medical triage decisions must always remain under qualified medical professionals.

7.2 Society

Efficient emergency patient management can improve the responsiveness of healthcare services and support better utilization of emergency department resources.

7.3 Healthcare Efficiency

A Priority Queue can help medical staff quickly identify the next patient according to predefined urgency levels.

This can reduce unnecessary manual sorting of waiting patients.

7.4 Resource Management

The system can help organize patient flow and assigned doctors.

Future versions could also consider doctor availability and specialization.

7.5 Equality and Fairness

The system applies the same priority rules to patients.

Patients with equal severity are treated according to their arrival sequence, preventing arbitrary ordering among patients with the same priority.

7.6 Scalability

The Binary Heap provides O(log n) insertion and deletion, making the system suitable for increasing patient loads.

7.7 Professional Responsibility

A real-world emergency management system must consider:

- Patient privacy
- Data security
- Medical validation
- Reliable software
- Fault handling
- Accurate severity assessment
- Human supervision

The proposed system is an educational demonstration of data-structure concepts and should not be considered a replacement for clinical triage.

8. Conclusion

The **Priority-Based Emergency Patient Management System** successfully demonstrates the application of Data Structures and Algorithms to an emergency healthcare scenario.

A conventional FIFO queue is not sufficient because it considers only arrival time and may cause critically ill patients to wait behind patients with less severe conditions.

Therefore, a **Priority Queue implemented using a Binary Max-Heap** was selected as the most appropriate data structure.

The system uses severity as the primary priority and arrival sequence as the secondary priority. This ensures that patients with critical conditions are treated first while maintaining FIFO order among patients with the same severity.

The major operations including insertion, next-patient selection, treatment/removal, searching, and updating were considered and analyzed.

The Binary Heap provides:

- **O(log n)** insertion
- **O(1)** next-patient access
- **O(log n)** deletion
- **O(n)** searching

- **O(n)** space complexity

The project demonstrates how an appropriate data structure can improve the efficiency, fairness, responsiveness, reliability, and scalability of emergency patient management.

9. Student Reflection

Through this assignment, I learned how Data Structures can be applied to solve a practical healthcare management problem. Initially, a normal FIFO queue appeared suitable because patients arrive continuously and can be processed according to their arrival order. However, after analyzing the emergency department requirements, I understood that arrival order alone cannot be used because patients have different levels of medical severity.

I learned that a Priority Queue is more appropriate because it allows critical patients to receive priority. I also learned that patients having the same severity should not lose their arrival order. Therefore, an arrival sequence was used as a secondary priority criterion.

Implementing the Binary Heap helped me understand priority queue operations such as insertion, deletion, heapify-up, and heapify-down. I also learned how time and space complexity can be used to evaluate whether a data structure is suitable for a real-world problem.

One of the main challenges was designing the priority comparison so that both severity and arrival order were correctly considered. This assignment helped me understand that selecting a data structure depends on the requirements of the application.

The project also helped me understand the importance of efficient healthcare systems and their relationship with SDG 3, SDG 9, SDG 10, and SDG 11.

Overall, this assignment improved my understanding of Priority Queues, Binary Heaps, complexity analysis, algorithm design, testing, and practical application of data structures.

10. References

1. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, **Introduction to Algorithms**, MIT Press.
2. Mark Allen Weiss, **Data Structures and Algorithm Analysis in C++**, Pearson.
3. Ellis Horowitz, Sartaj Sahni, and Susan Anderson-Freed, **Fundamentals of Data Structures**.

4. Bjarne Stroustrup, **The C++ Programming Language**, Addison-Wesley.